

AI 3D Studio Project Blueprint

Comprehensive codebase documentation covering architecture, call graphs, dependency analysis, API reference, security review, and debugging guide for the full-stack AI-powered 3D model generation platform.

326

SOURCE FILES

59

DIRECTORIES

20

DOCUMENT SECTIONS

60+

API ENDPOINTS

13

AI PROVIDERS

Stack: Next.js 16 + FastAPI + Celery + PostgreSQL + Redis

AI Models: Hunyuan3D, TRELIS, TripoSR, InstantMesh, AniGen

Analysis Date: August 2025

v3.3.1 Analysis

Table of Contents

1. Executive Summary	4
1.1 Architecture Overview	4
1.2 Technology Stack	4
1.3 Main Components	5
2. Complete Folder Structure	6
3. Complete File Index	8
3.1 Backend Core Files	8
3.2 Backend API Files	9
3.3 Provider Files	10
3.4 Worker and Runtime Files	11
3.5 Frontend Core Files	12
3.6 Frontend Feature Files	14
4. Architecture Diagrams	15
4.1 Overall System Architecture	15
4.2 Frontend Architecture	16
4.3 Backend Architecture	16
4.4 Storage Architecture	17
5. Complete Call Graph	17
5.1 Application Startup Sequence	18
5.2 Generation Request Flow	18
5.3 Model Download and Installation Flow	19
5.4 VRAM Management and Model Loading Flow	20
6. Dependency Graph	21
6.1 Python Backend Dependencies	21
6.2 Frontend npm Dependencies	22
6.3 Circular and Hidden Dependencies	23

7. Storage Architecture	23
8. Runtime Architecture	24
8.1 Runtime Engine	24
8.2 VRAM Allocation Tracker	25
8.3 Per-Model Virtual Environment System	25
9. Model Lifecycle	25
10. API Documentation	26
11. Frontend Architecture	29
11.1 Pages and Routing	30
11.2 State Management	30
11.3 Component Hierarchy	30
11.4 API Client Architecture	31
12. Docker and Deployment Architecture	31
12.1 Native Deployment (Primary)	32
12.2 Docker Packaging (Secondary)	32
12.3 Environment Variables	32
13. Startup Sequence	33
14. Generation Pipeline	34
15. Error Propagation	34
16. Debugging Guide	35
16.1 Download Failures	36
16.2 GPU / CUDA Issues	36
16.3 API / Backend Issues	36
16.4 Worker / Celery Issues	37
16.5 Frontend Issues	37
17. Dead Code Analysis	37
18. Refactoring Opportunities	38

18.1 Architecture Smells	38
18.2 Tight Coupling	39
18.3 Code Duplication	39
18.4 Hidden State	39
18.5 Recommended Improvements	40
19. Security Review	40
20. Appendix	41
20.1 Glossary	41
20.2 File Cross-Reference Index	42
20.3 Version History	44
20.4 Known Bugs	44

1. Executive Summary

AI 3D Studio is a full-stack, GPU-accelerated platform for AI-powered 3D model generation, management, texture editing, rigging, and rendering. The system enables users to generate 3D models from text descriptions or single images using state-of-the-art AI models including Hunyuan3D-2.1, TRELIS, TripoSR, and InstantMesh. It provides a comprehensive workspace with a layered project system, real-time 3D preview via React Three Fiber, and a sophisticated admin panel for system monitoring, model management, and download orchestration.

The architecture follows a hybrid approach: a Next.js 16 frontend (React 19, TypeScript, Tailwind CSS 4) communicates with a FastAPI (Python 3.12+) backend through a Next.js API proxy. The backend uses SQLAlchemy 2 with PostgreSQL (and SQLite fallback) for persistence, Redis for task queuing and pub/sub messaging, and Celery workers for asynchronous job execution including 3D model generation, texture application, and auto-rigging via AniGen. Each AI model is installed in its own isolated Python virtual environment under the `third_party` directory, preventing dependency conflicts between models that may require different versions of PyTorch, Transformers, or Diffusers.

1.1 Architecture Overview

The system is organized into five primary layers: (1) the Presentation Layer (Next.js with React 19, Framer Motion, React Three Fiber, Zustand state management, and shadcn/ui components); (2) the API Gateway Layer (Next.js catch-all route that proxies all `/api/v1/*` requests to the Python backend, handling multipart uploads and SSE streaming); (3) the Application Layer (FastAPI with Pydantic v2 validation, route-level error handling, and middleware for request timing and ID tracking); (4) the Worker Layer (Celery workers with Redis broker for GPU-bound generation tasks, download orchestration, and health monitoring); and (5) the Infrastructure Layer (PostgreSQL/SQLite, Redis, optional Blender integration for post-processing, and a VRAM allocation tracker with LRU eviction).

1.2 Technology Stack

Layer	Technology	Version	Purpose
Frontend Framework	Next.js (App Router)	16.2	SSR/SSG, routing, API proxy
UI Library	React	19.2	Component rendering
Language (Frontend)	TypeScript	5.9	Type-safe frontend code
Styling	Tailwind CSS	4.1	Utility-first CSS
UI Components	shadcn/ui (New York)	Latest	40+ accessible primitives

3D Rendering	React Three Fiber + Drei	9.6 / 10.7	WebGL 3D viewer
State Management	Zustand + React Query	5.0 / 5.101	Client + server state
Animation	Framer Motion	12.42	UI animations/transitions
Backend Framework	FastAPI	0.115	REST API, async handlers
Language (Backend)	Python	3.12+	Server-side logic
ORM	SQLAlchemy	2.0	Database abstraction
Database	PostgreSQL / SQLite	16 / 3.x	Primary / fallback storage
Task Queue	Celery + Redis	5.4 / 7.x	Async job processing
Migration	Alembic	1.14	Database schema migrations
AI Models	Hunyuan3D, TRELIS, TripoSR	Various	3D generation inference
Package Manager (Py)	uv	Latest	Fast Python package management

Table 1.1 - Complete Technology Stack

1.3 Main Components

The system is composed of several major subsystems that work together to deliver the end-to-end 3D model generation experience. The **Workspace** is the primary user interface, providing a sidebar-based creative workspace with tabs for 3D Generation, Segmentation, Rigging, Remesh, Texture Generation, Asset Management, and Settings. The **Generation Engine** handles the core AI inference pipeline, from prompt enhancement through model loading, GPU allocation, mesh generation, texture application, and optional auto-rigging. The **Model Manager** provides a unified interface for discovering, downloading, installing, and managing AI models from multiple providers (HuggingFace, GitHub, ModelScope, NVIDIA NGC, CivitAI). The **Admin Panel** offers real-time system monitoring, log streaming, terminal access, job queue management, and storage analytics. The **Runtime Engine** manages the lifecycle of AI model providers, including installation verification, GPU memory tracking, and automatic model unloading to prevent VRAM exhaustion.

High-level workflow: A user enters a text prompt or uploads a reference image in the Workspace. The frontend validates input, optionally uploads the image via the Upload API, and submits a GenerationRequest to the backend. The backend creates a GenerationJob record in the database, enqueues a Celery task, and returns the job ID. The Celery worker picks up the task, optionally enhances the prompt via OpenAI, loads the selected AI provider, allocates GPU memory via the VRAM tracker, runs inference, generates the 3D mesh, optionally applies textures and rigging, renders a thumbnail,

computes mesh statistics, and updates the job record with results. The frontend polls or receives SSE events for real-time progress, and displays the completed 3D model in the built-in Three.js viewer for inspection and download.

2. Complete Folder Structure

The project root contains 326 files across 59 directories. The structure follows a hybrid monorepo pattern where the Next.js frontend and Python backend coexist in the same repository. This section provides a comprehensive directory-by-directory breakdown explaining the purpose, responsibilities, contents, dependencies, and important notes for every folder in the project.

Directory	Purpose	Key Contents	Dependents
app/	Next.js App Router pages and API routes	page.tsx, layout.tsx, globals.css, 8 page routes, 2 API routes	All frontend features
app/api/v1/[...path]/	Backend API proxy (catch-all route)	route.ts - forwards requests to Python backend	All services/
app/static/[...path]/	Static file proxy for generated models	route.ts - serves backend static files	Viewer components
backend/	Python FastAPI backend application	app/, runtime/, tests/, requirements.txt	Frontend via API proxy
backend/app/	FastAPI application package	main.py, config.py, database.py	All backend modules
backend/app/api/v1/	REST API endpoint definitions (16 files)	generation.py, jobs.py, runtime.py, admin.py, etc.	Frontend services/
backend/app/core/	Core business logic layer	providers/, managers/, registry/, blender/	API routes, workers
backend/app/core/providers/	AI model provider implementations (17 files)	hunyuan3d_local.py, trellis_local.py, registry.py, base.py	Runtime engine, tasks
backend/app/core/managers/	System management components	vram_tracker.py, health_manager.py, plugin_manager.py	API routes, workers
backend/app/core/registry/	Model registry database operations	model_registry.py	models_api.py, pipelines.py
backend/app/core/blender/	Blender integration for post-processing	pipeline.py, scripts/process_mesh.py	tasks.py, project.py
backend/app/core/download_manager/	Download queue management system	downloader.py, manifest.py, provider_base.py	download.py API

Directory	Purpose	Key Contents	Dependents
backend/app/core/download/	Download utilities (mirror fallback, checksum)	mirror_fallback.py, checksum_validator.py	download_manager/
backend/app/core/installer/	Model installation and dependency resolution	plugin_installer.py, dependency_resolver.py	runtime/installer.py
backend/app/models/	SQLAlchemy ORM models	job.py (GenerationJob, VramAuditLog), registry.py	All API routes, workers
backend/app/schemas/	Pydantic validation schemas	generation.py, manifest.py	API routes
backend/app/workers/	Celery task definitions	tasks.py (generate_3d_model), celery_app.py, download_workers.py	Celery broker
backend/app/utils/	Shared utility functions	response.py (success/error/paginated), storage.py	All API routes
backend/runtime/	Runtime engine for model lifecycle	engine.py, installer.py, gpu.py, storage.py, health.py	backend/app/core/
backend/alembic/	Database migration scripts	env.py, versions/0001_initial.py	start.sh
components/	Shared React UI components	motion/, premium/, ui/, ux/, AppearanceProvider.tsx	All features/
components/ui/	shadcn/ui component library (40+ files)	button.tsx, card.tsx, dialog.tsx, tabs.tsx, etc.	All features, app/
components/motion/	Framer Motion animation components (14 files)	FadeIn, GlassCard, MagneticButton, PageTransition, etc.	Landing, workspace
components/premium/	Premium design components (7 files)	GlassCard, NeonButton, ProgressBar, Spinner, etc.	Workspace, admin
features/workspace/	Main workspace UI (primary user interface)	WorkspaceShell.tsx, LeftSidebar.tsx, PromptInput.tsx	app/page.tsx
features/workspace/new-ui/	Creative workspace layout with tab panels (15 files)	CreativeWorkspaceLayout.tsx, ThreeDGenerationTab.tsx	WorkspaceShell
features/workspace/viewer/	3D model viewer (React Three Fiber)	ThreeDViewer.tsx, ViewerScene.tsx, ViewerToolbar.tsx	WorkspaceShell
features/admin/	Admin panel UI (legacy, merged into settings)	AdminShell.tsx, tabs/ (12 tab components)	app/admin/page.tsx
features/model-manager/	Model management UI	ModelCard.tsx, tabs/ (6 tabs), components/ (4 components)	Settings page

Directory	Purpose	Key Contents	Dependents
features/settings/	Settings page sections (11 sections)	AppearanceSection.tsx, PipelinesSection.tsx, etc.	app/settings/page.tsx
features/texture/	Texture generation page (mock)	TextureShell.tsx	app/texture/page.tsx
features/render/	Render studio page (mock)	RenderShell.tsx	app/render/page.tsx
hooks/	Custom React hooks (7 files)	useGeneration.ts, useTaskManager.ts, useBackendData.ts	All components
services/	API communication layer (5 files)	apiClient.ts, generationService.ts, runtimeService.ts	All hooks, stores
stores/	Zustand state management (5 files)	useAppStore.ts (primary), useThemeStore.ts	All components, hooks
lib/	Core utilities and providers	providers.tsx, utils.ts, next-compat-*.tsx (4 files)	app/layout.tsx
types/	TypeScript type definitions	index.ts (50+ types), new-ui.ts	All TS files
constants/	Application-wide constants	index.ts (presets, formats, suggestions)	Features, components
scripts/	Deployment and management shell scripts	setup.sh, start.sh, stop.sh, colab.sh, cloudflare.sh	manager.sh, ops
Docs/	Project documentation (6 files)	README.md, architecture.md, api-documentation.md	Developers

Table 2.1 - Complete Directory Structure with Dependencies

3. Complete File Index

This section catalogs every important file in the repository with detailed metadata including purpose, responsibility, key exports, dependencies, risk level, and modification safety assessment. Files are organized by their directory location. Risk levels are classified as: Critical (breaking changes cause complete system failure), High (significant functionality loss), Medium (localized impact), and Low (cosmetic or easily replaceable).

3.1 Backend Core Files

File Path	Purpose	Risk	Safe to Modify?
backend/app/main.py	FastAPI app entry point with lifespan, middleware, CORS, startup diagnostics, and global error handler	Critical	No
backend/app/config.py	Pydantic Settings class with 35+ configuration fields covering DB, Redis, AI providers, GPU, storage, Blender	Critical	Yes (carefully)
backend/app/database.py	SQLAlchemy async/sync engine setup, session factories, Base class, get_db dependency	Critical	No
backend/app/models/job.py	GenerationJob and VramAuditLog ORM models with UUID PKs, status enums, config snapshots, result metadata	High	Yes (additive)
backend/app/models/registry.py	DownloadQueue, DownloadChunk, InstalledModel, ModelCapability, ModelDependency ORM models	High	Yes (additive)
backend/app/schemas/generation.py	Pydantic GenerationRequest with validators (2000 char prompt, 6 modes, 5 quality levels), JobResult, ProgressEvent	High	Yes
backend/app/schemas/manifest.py	ManifestSchema - complete model manifest with metadata, capabilities (14 boolean flags), dependencies, download sources	Medium	Yes
backend/app/utis/response.py	Standardized API response helpers: success(), error(), paginated(). Has known syntax bug on line 15	High	Yes (fix bug)
backend/app/utis/storage.py	File upload handler (validate type/size, UUID filename, aiofiles write), model output dir helper	Medium	Yes

Table 3.1 - Backend Core Files

3.2 Backend API Files

File Path	Purpose	Endpoints	Risk
backend/app/api/v1/generation.py	3D generation lifecycle: create job, poll status, get history, estimate cost	4 endpoints	Critical
backend/app/api/v1/jobs.py	Job CRUD operations: list, get detail, delete (with file cleanup)	3 endpoints	High
backend/app/api/v1/runtime.py	Runtime engine management: status, install, switch provider, verify, clear VRAM	15+ endpoints	Critical
backend/app/api/v1/admin.py	Full admin panel API (1811 lines): logging, terminal, download progress, SSE streams, system overview	30+ endpoints	Critical
backend/app/api/v1/health.py	Health check aggregating DB, Redis, storage, runtime engine status	1 endpoint	Medium

File Path	Purpose	Endpoints	Risk
backend/app/api/v1/upload.py	Image upload (PNG/JPG/WEBP, 256-8192px, 50MB) and model upload (GLB/GLTF, 100MB)	3 endpoints	Medium
backend/app/api/v1/hf_token.py	HuggingFace token CRUD with file-based storage and HfApi.whoami() verification	4 endpoints	Medium
backend/app/api/v1/rigging.py	AniGen rigging pipeline: trigger rigging task, poll status via Redis + Celery	2 endpoints	High
backend/app/api/v1/project.py	Project export with layer toggles (texture, rigging, animation, LOD) via Blender pipeline	1 endpoint	Medium
backend/app/api/v1/pipelines.py	Pipeline capability matrix: workspace models, feature toggles, workspace types	4 endpoints	Medium
backend/app/api/v1/plugin_manager.py	Model plugin management: install/uninstall/load/unload via PluginManager	2 endpoints	Medium
backend/app/api/v1/discover.py	Model discovery: multi-provider search, featured models, categories, provider list	4 endpoints	Low
backend/app/api/v1/download.py	Download queue management: start, pause, resume, cancel, batch, statistics	10+ endpoints	High
backend/app/api/v1/models_api.py	Model CRUD: list, installed, get detail, delete, repair, health check, verify all	8 endpoints	High
backend/app/api/v1/system.py	System info: compatibility check, GPU info, dependency verification, storage info, statistics	8 endpoints	Medium
backend/app/api/v1/settings.py	In-memory appearance and workspace config management	4 endpoints	Low

Table 3.2 - Backend API Files

3.3 Provider Files

File Path	Provider	Type	VRAM	Status
core/providers/base.py	Base classes + _add_model_env()	Framework	N/A	Active
core/providers/registry.py	Provider registry, RuntimeProviderRegistry	Framework	N/A	Active
core/providers/mock.py	MockProvider - placeholder generation	Mock	0	Active
core/providers/hunyuan3d_local.py	Hunyuan3D-2.1 / Hunyuan3D-2 local inference	Local	16/12 GB	Active

File Path	Provider	Type	VRAM	Status
core/providers/trellis_local.py	TRELLIS image-to-3D local inference	Local	~8 GB	Active
core/providers/triposr_local.py	TripoSR image-to-3D local inference	Local	~4 GB	Active
core/providers/instant_mesh.py	InstantMesh API wrapper (stub)	API	N/A	Stub
core/providers/anigen_provider.py	AniGen character rigging	Local	6.2 GB	Active
core/providers/triposg_provider.py	TripoSG mesh generation	Mock	12 GB	Mock
core/providers/triposf_provider.py	TripoSF mesh generation (has syntax bug)	Mock	12 GB	Mock (Broken)
core/providers/unirig_provider.py	UniRig rigging	Mock	8 GB	Mock
core/providers/holopart_provider.py	HoloPart segmentation	Mock	8 GB	Mock
core/providers/detailgen3d.py	DetailGen3D mesh detailing	Mock	4 GB	Mock
core/providers/hunyuan3d.py	Hunyuan3D API wrapper (stub)	API	N/A	Stub
core/providers/trellis.py	TRELLIS API wrapper (stub)	API	N/A	Stub
core/providers/triposr.py	TripoSR API wrapper (stub)	API	N/A	Stub
core/providers/huggingface_provider.py	HuggingFace download provider (stub)	Download	N/A	Stub
core/providers/github_provider.py	GitHub Releases download provider	Download	N/A	Active
core/providers/modelscope_provider.py	ModelScope download provider	Download	N/A	Active
core/providers/nvidia_ngc_provider.py	NVIDIA NGC catalog download provider	Download	N/A	Active
core/providers/civitai_provider.py	CivitAI download provider with license check	Download	N/A	Active

Table 3.3 - AI Provider Files

3.4 Worker and Runtime Files

File Path	Purpose	Key Functions/Tasks	Risk
backend/app/workers/celery_app.py	Celery configuration with CUDA normalization, solo pool, JSON serialization, task registration	App initialization, beat schedule	Critical
backend/app/workers/tasks.py	Core generation pipeline: prompt enhance, provider load, generate, Blender post-process, thumbnail, stats	generate_3d_model, anigen_rig_task	Critical
backend/app/workers/vram_health_worker.py	30-second periodic VRAM monitoring, auto-eviction at 90% pressure, Redis metrics	check_vram_health	High
backend/app/workers/download_workers.py	Download queue execution, resumable downloads with chunk tracking	download_model, process_queue	High
backend/app/workers/health_workers.py	System health check task for admin panel	get_system_health	Medium
backend/app/workers/installation_workers.py	Model install/uninstall/repair tasks	uninstall_model, repair_model	High
backend/runtime/engine.py	RuntimeEngine: provider load/unload, health check, initialization	load_provider, unload_provider, health	Critical
backend/runtime/installer.py	RuntimeInstaller: full install, verify, clone repos, install deps, download weights, get status	full_install, get_install_status	Critical
backend/runtime/gpu.py	GPU detection, VRAM usage, CUDA cache management, device selection	get_gpu_info, get_vram_usage, empty_cuda_cache	High
backend/runtime/storage.py	Storage configuration: directory structure, disk usage, model paths, venv paths	ensure_dirs, get_repo_path, get_weight_path	High
backend/runtime/health.py	Runtime health checks: GPU, CUDA, Blender, providers, repos, weights, services	check_all	Medium
backend/runtime/platform_detection.py	Platform detection: Colab, Codespaces, WSL, VPS	detect_platform	Low

Table 3.4 - Worker and Runtime Files

3.5 Frontend Core Files

File Path	Purpose	Key Exports	Risk
app/layout.tsx	Root layout with fonts, providers, ambient glow effects	Providers, AppearanceProvider	High

File Path	Purpose	Key Exports	Risk
app/page.tsx	Root page - renders WorkspaceShell	WorkspaceShell	Medium
app/api/v1/[...path]/route.ts	Backend API proxy with SSE support, multipart forwarding	GET/POST/PUT/DELETE handlers	Critical
app/settings/page.tsx	Unified settings page with 17 sections, 5 groups, sidebar navigation	Settings layout	Medium
stores/useAppStore.ts	PRIMARY Zustand store: generation, UI, tasks, downloads, project, layers	useAppStore	Critical
stores/useThemeStore.ts	Theming: 10 presets, 50+ CSS vars, 13 animation toggles, glass effects	useThemeStore	High
services/apiClient.ts	HTTP client: caching, retry, circuit breaker, SSE, file upload	apiClient	Critical
services/generationService.ts	Generation API: start, poll, cancel with concurrent safety	generationService	Critical
services/runtimeService.ts	Runtime management API: options, status, install, switch, HF token	runtimeService	High
services/adminService.ts	Admin API: overview, logs, terminal, queue, models, pipelines	adminService	High
hooks/useGeneration.ts	Core generation orchestration: validate, upload, submit, progress, events	useGeneration	Critical
hooks/useTaskManager.ts	Background task polling, lifecycle management, reconnect on refresh	useTaskManager	High
hooks/useBackendData.ts	Data-fetching hooks: status, history, models, SSE events	useBackendStatus, useRuntimeOptions	High
hooks/useEventBus.ts	Typed singleton event bus for cross-component communication (20 events)	getEventBus, useEventBus	Medium
lib/providers.tsx	Root providers: QueryClientProvider, Sonner Toaster, theme integration	Providers	High
lib/utls.ts	Tailwind class merge utility (clsx + twMerge)	cn	Low
components/AppearanceProvider.tsx	Writes 50+ CSS custom properties from ThemeConfig to DOM	AppearanceProvider	High
types/index.ts	50+ type definitions: jobs, UI, admin, pipelines, projects, layers	All types	High

Table 3.5 - Frontend Core Files

3.6 Frontend Feature Files

File Path	Purpose	Size	Risk
features/workspace/WorkspaceShell.tsx	Main workspace container, fullscreen mode, dynamic 3D viewer import	Medium	High
features/workspace/new-ui/CreativeWorkspaceLayout.tsx	PRIMARY workspace: 12 sidebar items, capability filtering, history management	Large	Critical
features/workspace/new-ui/ThreedGenerationTab.tsx	MAIN generation UI: model selector, prompt, quality, 3D preview, history (69KB)	69 KB	Critical
features/workspace/new-ui/WorkspaceTab.tsx	Dashboard/home: recent generations, quick actions	Medium	Medium
features/workspace/new-ui/TextureGenTab.tsx	Texture generation controls with material layer editing	Medium	Medium
features/workspace/new-ui/ExportDialog.tsx	Export format selection (GLB/OBJ/FBX)	Small	Low
features/workspace/viewer/ThreedViewer.tsx	React Three Fiber Canvas with Stats, ViewerToolbar, placeholder	Medium	High
features/workspace/viewer/ViewerScene.tsx	Three.js scene: lighting, environment, grid, model loading	Medium	High
features/workspace/PromptInput.tsx	Text prompt input with suggestions and image upload	Small	Medium
features/workspace/ModelSelector.tsx	AI model selector dropdown with provider info	Small	Medium
features/workspace/GenerateButton.tsx	Main generation trigger with loading states	Small	Medium
features/workspace/JobProgressMonitor.tsx	Real-time job progress with stage indicators	Small	Medium
features/admin/AdminShell.tsx	Legacy admin panel shell (deprecated, redirects to settings)	Medium	Low
features/admin/tabs/OverviewTab.tsx	System dashboard: GPU, CPU, RAM, storage metrics	Medium	Medium
features/admin/tabs/LogsTab.tsx	Live log viewer with SSE streaming	Medium	Medium
features/admin/tabs/TerminalTab.tsx	Web terminal for running commands	Small	High
features/settings/sections/AppearanceSection.tsx	Full theme editor: 10 presets, all ThemeConfig fields, live preview	Large	Medium
features/settings/sections/PipelinesSection.tsx	Pipeline dashboard with feature flags, model compatibility	Large	Medium
features/texture/TextureShell.tsx	PBR material editor (mock - no real backend)	Medium	Low

File Path	Purpose	Size	Risk
features/render/RenderShell.tsx	Render studio (mock - hardcoded queue data)	Medium	Low

Table 3.6 - Frontend Feature Files

4. Architecture Diagrams

This section presents the key architectural diagrams of the AI 3D Studio system. Diagrams are described using Mermaid notation for clarity and can be rendered using any Mermaid-compatible tool. The architecture follows a layered pattern with clear separation between the frontend presentation layer, the API gateway, the application backend, the worker layer, and the infrastructure services.

4.1 Overall System Architecture

The overall system architecture shows how all major components interact. The browser communicates with the Next.js frontend, which proxies API requests to the FastAPI backend. The backend uses Celery workers with Redis as a message broker for asynchronous tasks. PostgreSQL serves as the primary database with SQLite as a fallback for cloud environments. AI model inference runs on GPU via isolated Python environments.

```
graph TB
    subgraph Browser [Browser - Chrome/Edge/Firefox]
        UI[React 19 UI Next.js 16]
        R3F[React Three Fiber 3D Viewer]
    end
    subgraph Frontend [Next.js Frontend :3000]
        AP[API Proxy app/api/v1/route.ts]
        SP[Static Proxy app/static/route.ts]
    end
    subgraph Backend [FastAPI Backend :8000]
        MW[Middleware Timing + Request ID]
        API[REST API Layer 16 Route Files]
        VAL[Pydantic Validation Schemas]
    end
    subgraph Workers [Celery Workers]
        GW[Generation Worker generate_3d_model]
        DW[Download Worker download_model]
        HW[Health Worker check_vram_health]
        IW[Install Worker install/repair]
    end
    subgraph AI [AI Model Providers]
        HY[Hunyuan3D-2.1 16GB VRAM]
        TR[TRELLIS 8GB VRAM]
        TS[TripoSR 4GB VRAM]
        AG[AniGen 6.2GB VRAM]
    end
    subgraph Infra [Infrastructure]
        PG[(PostgreSQL / SQLite)]
        RD[(Redis)]
        FS[(File Storage storage/)]
        BL[Blender 4.2 Post-Processing]
    end
    UI --> AP
    AP --> MW
    MW --> API
    API --> VAL
    VAL --> R3F
    R3F --> SP
    SP --> FS
    FS --> AP
    AP --> AI
    AI --> DW
    DW --> API
    API --> HW
    HW --> IW
    IW --> GW
    GW --> BL
    BL --> FS
    FS --> PG
    PG --> RD
    RD --> API
    API --> HY
    HY --> TR
    TR --> TS
    TS --> AG
    AG --> BL
```

Diagram 4.1 - Overall System Architecture (Mermaid)

4.2 Frontend Architecture

The frontend architecture is organized into a clear hierarchy. The root layout provides fonts, theme providers, and ambient visual effects. The primary store (useAppStore) is a single Zustand store with localStorage persistence that serves as the source of truth for generation config, UI state, tasks, downloads, and project layers. Three proxy stores (useUIStore, useGenerationStore, useProjectStore) provide domain-specific APIs that delegate to the primary store. Services handle API communication with caching, retry, and circuit breaker patterns. The event bus enables cross-component communication without prop drilling.

```
graph TB
    subgraph Pages [Pages]
        HP[Home /] --> WS[WorkspaceShell]
        ST[Settings /settings]
        AD[Admin /admin]
        GN[Generate /generate]
        TX[Texture /texture]
        RN[Render /render]
    end
    subgraph Workspace [Workspace]
        CWL[CreativeWorkspaceLayout]
        T3D[ThreeDGenerationTab]
        V3D[ThreeDViewer]
        WS2[WorkspaceTab]
        FT[TextureGenTab]
        RM[RemeshTab]
        SG[SegmentationTab]
        RA[RiggingAnimationTab]
        EX[ExportDialog]
        LV[LayerVisibilityPanel]
    end
    subgraph StateManagement [State Management]
        AS[useAppStore]
        TS[useThemeStore]
        US[useUIStore]
        GS[useGenerationStore]
        PS[useProjectStore]
    end
    subgraph Services [API Services]
        AC[apiClient]
        GES[generationService]
        RTS[runtimeService]
        ADS[adminService]
        UPS[uploadService]
        CAS[cacheService]
    end
    subgraph Hooks [Custom Hooks]
        UG[useGeneration]
        UTM[useTaskManager]
        UBD[useBackendData]
        UEB[useEventBus]
        UAS[useAutoSave]
    end
    CWL --> T3D
    T3D --> V3D
    V3D --> UG
    UG --> GES
    GES --> AC
    AC --> UPS
    UPS --> AC
    T3D --> UTM
    UTM --> UBD
    UBD --> AC
    AC --> US
    US --> AS
    T3D --> GS
    GS --> AS
```

Diagram 4.2 - Frontend Architecture (Mermaid)

4.3 Backend Architecture

```
graph TB
    subgraph Entry [Application Entry]
        MAIN[main.py]
    end
    subgraph API [API Layer - 16 Route Modules]
        GEN[generation.py]
        JOBS[jobs.py]
        RT[runtime.py]
        ADM[admin.py]
        UP[upload.py]
        HF[hf_token.py]
        PL[pipelines.py]
    end
    GEN --> JOBS
    JOBS --> RT
    RT --> ADM
    ADM --> UP
    UP --> HF
    HF --> PL
```

```

4 endpoints] SYS[system.py
8 endpoints] DIS[discover.py
4 endpoints] DL[download.py
10+ endpoints] MOD[models_api.py
8 endpoints] RIG[rigging.py
2 endpoints] PROJ[project.py
1 endpoint] SET[settings.py
4 endpoints] HEA[health.py
1 endpoint] PM[plugin_manager.py
2 endpoints] end subgraph Core[Core Business Logic] PROV[Provider Registry
13 Providers] MGR[Managers
VRAM/Health/Plugin/Env] REG[Model Registry
DB Operations] BLND[Blender Pipeline
Post-Processing] PMT[Prompt Enhancer
OpenAI GPT] MESH[Mesh Processor
Trimesh + PyMeshLab] CAP[Capability Matrix
Feature Gating] INST[Installer
Dependency Resolution] DLM[Download Manager
Queue + Mirrors] end subgraph Data[Data Layer] ORM[SQLAlchemy 2 ORM
6 Models] SCH[Pydantic Schemas
Request/Response] ALE[Alembic Migrations] end subgraph Workers[Celery Workers]
T1[tasks.py
generate_3d_model] T2[download_workers.py] T3[vram_health_worker.py]
T4[health_workers.py] T5[installation_workers.py] end MAIN --> API --> Core -->
Data API --> Workers Workers --> Core

```

Diagram 4.3 - Backend Architecture (Mermaid)

4.4 Storage Architecture

```

graph TB
    subgraph Root [Project Root]
        TP[third_party/
        AI Model Repos + Venvs + Weights]
        ST[storage/
        User Generated Content]
        RC[runtime_cache/
        Install State]
        LG[logs/
        Application Logs]
        DB[*.db / *.sqlite
        SQLite Fallback]
    end
    subgraph ThirdParty [third_party/ Structure]
        H3D[Hunyuan3D-2/
        repo/ + .env/ + weights/]
        TRL[TRELLIS/
        repo/ + .env/ + weights/]
        TSR[TripoSR/
        repo/ + .env/ + weights/]
        IM[InstantMesh/
        repo/ + .env/ + weights/]
    end
    subgraph StorageFiles [storage/ Structure]
        UP[uploads/
        User Images]
        OUT[outputs/
        Generated Models]
        THU[thumbnails/
        Model Previews]
        TMP[temp/
        Processing Files]
    end
    TP --> ThirdParty
    ST --> StorageFiles
    subgraph External
        PG_DB[(PostgreSQL
        Jobs, Models, Downloads)]
        REDIS[(Redis
        Queue, Pub/Sub, Cache)]
    end

```

Diagram 4.4 - Storage Architecture (Mermaid)

5. Complete Call Graph

This section traces every major execution path through the system, documenting the file, class, function, return value, and possible exceptions at each step. Call graphs are presented in a linear sequence showing the complete path from user action to final result.

5.1 Application Startup Sequence

Step	File	Function/Action	Return/Effect	Possible Exceptions
1	scripts/start.sh	Start PostgreSQL (or fallback to SQLite)	DB connection string	Auth failure, PID missing
2	scripts/start.sh	Start Redis (or fallback to in-process broker)	Redis URL	Port conflict
3	scripts/start.sh	Run Alembic migrations (alembic upgrade head)	Migration applied	Migration conflict
4	scripts/start.sh	Start Backend API (uvicorn backend.app.main:app)	Process PID	Import error, port busy
5	backend/app/main.py	lifespan(): normalize CUDA_VISIBLE_DEVICES	Env var set	None
6	backend/app/main.py	lifespan(): log_startup_diagnostics()	Diagnostics logged	GPU detection failure
7	backend/app/main.py	lifespan(): ensure storage dirs	Dirs created	Permission error
8	backend/app/main.py	lifespan(): init RuntimeEngine	Engine ready	GPU init failure
9	backend/app/main.py	lifespan(): test DB connection + create tables	Tables created	Connection refused
10	backend/app/main.py	lifespan(): test Redis connection	Redis ping OK	Connection refused
11	scripts/start.sh	Start Celery Worker (celery -A workers.celery_app)	Worker PID	Import error, broker failure
12	scripts/start.sh	Start Frontend (next start / standalone server)	Frontend PID	Build missing, port busy

Table 5.1 - Application Startup Call Graph

5.2 Generation Request Flow

Step	File	Function	Return	Exceptions
1	hooks/useGeneration.ts	generate(): validate prompt/image	Validation result	Empty prompt, invalid image
2	services/uploadService.ts	upload(): POST /api/v1/upload/image	UploadResponse	Size limit, format error
3	services/generationService.ts	startGeneration(): POST /api/v1/generation	Job ID (UUID)	Network error, 4xx/5xx
4	app/api/v1/[...path]/route.ts	POST handler: forward to BACKEND_URL	Backend response	Backend unreachable
5	backend/app/api/v1/generation.py	create_generation(): validate request, create job	GenerationJob	Validation error, DB error
6	backend/app/api/v1/generation.py	Enqueue Celery task: generate_3d_model.delay()	Task ID	Broker failure
7	backend/app/workers/tasks.py	generate_3d_model(): enhance prompt (optional)	Enhanced prompt	OpenAI API error
8	backend/app/workers/tasks.py	generate_3d_model(): get_provider()	Provider instance	Provider not found
9	backend/app/core/provider_registry.py	get_provider(): dynamic import + instantiation	BaseProvider	Import error, VRAM fail
10	backend/app/core/provider_registry.py	_add_model_env(): prepend venv to sys.path	Modified sys.path	Venv not found
11	backend/app/workers/tasks.py	provider.generate(prompt, image, config)	ProviderResult	GPU OOM, CUDA error
12	backend/app/workers/tasks.py	Blender post-processing (texture, rig, export)	Processed GLB	Blender timeout
13	backend/app/workers/tasks.py	render_thumbnail() + get_mesh_stats()	Thumbnail path, stats	Trimesh error
14	backend/app/workers/tasks.py	Update job record: status=completed, results	Updated job	DB write error
15	services/generationService.ts	Poll /api/v1/generation/{id}/status	Job status + result	Timeout, 5xx
16	hooks/useGeneration.ts	Update store, emit generation:completed event	UI updated	None

Table 5.2 - Generation Request Call Graph

5.3 Model Download and Installation Flow

Step	File	Function	Return	Exceptions
1	features/admin/tabs/ModelsTab.tsx	User clicks Install on model card	Action dispatch	None
2	services/runtimeService.ts	install(model_id): POST /api/v1/runtime/install	Install initiated	Network error
3	backend/app/api/v1/runtime.py	install(): RuntimeInstaller.full_install()	Background task	Invalid model
4	backend/runtime/installer.py	full_install(): verify_system, clone_repo, install_deps, download_weights	Install status	Git clone fail, pip error
5	backend/runtime/storage.py	ensure_dirs(): create third_party//repo/, .venv/, weights/	Dirs created	Permission error
6	backend/runtime/installer.py	install_repo_deps(): uv pip install in model venv	Dependencies installed	Resolution failure
7	backend/runtime/installer.py	download_weights(): huggingface_hub download	Weights downloaded	Network error, disk full
8	backend/app/api/v1/admin.py	SSE stream install progress to frontend	Progress events	Connection drop

Table 5.3 - Model Download and Installation Call Graph

5.4 VRAM Management and Model Loading Flow

Step	File	Function	Return	Exceptions
1	backend/app/workers/tasks.py	generate_3d_model(): load provider	Provider instance	Import failure
2	backend/runtime/engine.py	RuntimeEngine.load_provider(name)	Loaded provider	Not found
3	backend/app/core/managers/vram_tracker.py	VRAMAllocationTracker.allocate(model_id, size_gb)	Allocation result	Insufficient VRAM
4	backend/app/core/managers/vram_tracker.py	If insufficient: auto-evict oldest model (LRU)	Eviction result	All models needed
5	backend/app/core/managers/vram_tracker.py	Write VramAuditLog entry (DB)	Audit record	DB write failure
6	backend/app/core/providers/hunyuan3d_local.py	_HunyuanBase._load_model(): verify GPU, load pipeline	Model loaded	CUDA OOM, model corrupt
7	backend/app/workers/tasks.py	provider.generate() - inference runs on GPU	ProviderResult	GPU timeout, driver crash

Step	File	Function	Return	Exceptions
8	backend/app/workers/tasks.py	engine.unload_provider() - release VRAM	Provider unloaded	None
9	backend/app/core/managers/vram_tracker.py	VRAMAllocationTracker.deallocate(model_id)	VRAM freed	Not allocated

Table 5.4 - VRAM Management Call Graph

6. Dependency Graph

This section analyzes the complete dependency graph of the project, covering Python backend imports, TypeScript frontend imports, runtime dependencies, circular dependencies, and potential issues.

6.1 Python Backend Dependencies

Package	Version	Purpose	Critical?
fastapi	0.115.6	Web framework - REST API	Yes
uvicorn	0.32.1	ASGI server	Yes
sqlalchemy	2.0.36	ORM / database	Yes
alembic	1.14.0	Database migrations	Yes
asyncpg	0.30.0	PostgreSQL async driver	Yes
aiosqlite	Latest	SQLite async driver (fallback)	Yes
redis	5.2.1	Redis client / Celery broker	Yes
celery	5.4.0	Distributed task queue	Yes
httpx	0.28.1	Async HTTP client	Medium
aiohttp	3.9.1	Async HTTP (GitHub provider)	Medium
Pillow	11.1.0	Image processing / validation	Medium
trimesh	4.5.3	Mesh processing / stats / cleanup	Medium
huggingface_hub	0.27.1	Model download / HfApi	Yes
openai	1.60.1	Prompt enhancement (optional)	Low
psutil	6.1.1	System resource monitoring	Medium
pynvml	Latest	NVIDIA GPU monitoring	Medium

Package	Version	Purpose	Critical?
pydantic	2.x	Settings / validation	Yes
pydantic-settings	Latest	.env configuration	Yes
torch (PyTorch)	CUDA 12.1	GPU tensor operations (per-model venv)	Yes

Table 6.1 - Python Backend Dependencies

6.2 Frontend npm Dependencies

Package	Version	Purpose	Status
next	16.2.11	React framework (App Router)	Active
react / react-dom	19.2.8	UI rendering	Active
three	0.185.1	3D graphics engine	Active
@react-three/fiber	9.6.1	React Three.js bindings	Active
@react-three/drei	10.7.7	R3F helpers (OrbitControls, Grid, etc.)	Active
zustand	5.0.14	State management	Active
@tanstack/react-query	5.101.2	Server state management	Active
framer-motion	12.42.2	Animation library	Active
@radix-ui/*	Various	Accessible UI primitives (22 packages)	Active
recharts	3.10.0	Chart library	Active
react-hook-form + @hookform/resolvers	7.82 / 5.2	Form validation	Active
sonner	Latest	Toast notifications	Active
react-dropzone	Latest	File drag-and-drop	Active
@google/genai	1.36.0	Google Gemini API client	Active
vite (devDep)	8.0.0	Build tool (UNUSED - project uses Next.js)	Dead
@tailwindcss/vite (devDep)	4.3.3	Vite Tailwind plugin (UNUSED)	Dead
@types/jquery (devDep)	4.0.1	Type declarations (no jQuery in project)	Dead
firebase-tools (devDep)	14.23	Firebase CLI (possible deployment)	Possibly unused

Table 6.2 - Frontend npm Dependencies

6.3 Circular and Hidden Dependencies

The codebase exhibits a layered dependency structure with minimal circular dependencies. However, several notable patterns exist. The **backend/app/core/providers/registry.py** module imports from **runtime.installer** at module level via a cached function, creating a potential circular dependency between the core layer and the runtime layer. This is mitigated by lazy evaluation through **lru_cache**. The **proxy store pattern** in the frontend (**useUIStore**, **useGenerationStore**, **useProjectStore** all delegating to **useAppStore**) creates a conceptual dependency cycle that is safe because it is unidirectional at runtime (proxies read from the primary store and write through **setState**).

Hidden dependencies include: (1) PyTorch is installed in each per-model venv rather than the main Python environment, making it invisible to standard dependency analysis tools; (2) Blender is an external system dependency invoked via subprocess, not a Python import; (3) the **BACKEND_URL** environment variable is read at request time (not build time) in the Next.js API proxy, creating a runtime dependency on Docker networking configuration; (4) the **AGENTS.md** file acts as a hidden dependency for AI coding assistants, containing project-specific conventions that affect code generation quality.

7. Storage Architecture

This section documents every storage location in the system, explaining who creates it, which file and function creates it, who reads and writes to it, and its lifecycle. The storage architecture follows a per-model isolation pattern where each AI model has its own directory containing the git repository, Python virtual environment, and model weights.

Path	Creator	Readers	Writers	Lifecycle
third_party/	runtime/storage.py:ensure_dirs()	All providers, installer	installer.py	Persistent, gitignored
third_party//repo/	runtime/installer.py:clone_repo()	Provider via _add_model_env()	Git clone	Updated via git pull
third_party//.venv/	runtime/installer.py:install_repo_deps()	Provider via _add_model_env()	uv venv + pip install	Recreated on reinstall
third_party//weights/	runtime/installer.py:download_weights()	Provider inference	huggingface_hub download	Downloaded once, cached
third_party//installing.lock	installer.py (start install)	start.sh (stale cleanup)	installer.py (remove on complete)	Temporary, 5-min stale threshold
storage/uploads/	backend/app/utils/storage.py:save_upload()	Static proxy, provider	Upload API	Persistent until job cleanup

Path	Creator	Readers	Writers	Lifecycle
storage/outputs//	workers/tasks.py (generation result)	Static proxy, frontend viewer	Celery worker	Persistent until manual delete
storage/thumbnails /	workers/tasks.py:render_thumbnail()	Frontend history list	Celery worker	Persistent
storage/temp/	Various processing functions	Workers	Workers	Cleaned periodically
runtime_cache/	runtime/installer.py: get_install_status()	Installer, API routes	Installer (JSON state files)	Persistent, gitignored
logs/	scripts/start.sh, admin log handler	Admin LogsTab, terminal	Admin _AdminLogHandler	Rotated at 5MB, 2000 ring buffer
*.db / *.sqlite	SQLAlchemy (SQLite fallback)	All DB operations	Alembic migrations	Persistent (fallback only)
PostgreSQL (ai3dstudio)	scripts/setup.sh:install_postgresql()	All DB operations	Alembic migrations	Persistent, backed up
Redis (ai3dstudio)	scripts/setup.sh:install_redis()	Celery, VRAM tracker, pub/sub	Celery workers, API	Ephemeral (in-memory)
.hf_token	backend/app/api/v1/ hf_token.py	download workers, installer	HF token API	Persistent, gitignored

Table 7.1 - Complete Storage Architecture

8. Runtime Architecture

The runtime architecture manages the complete lifecycle of AI model providers, from installation through inference to cleanup. It is implemented across the **backend/runtime/** module and the **backend/app/core/managers/** directory. The key components are the RuntimeEngine (provider load/unload), RuntimeInstaller (model installation), VRAMAllocationTracker (GPU memory management), HealthManager (model health checks), and the per-model virtual environment system.

8.1 Runtime Engine

The **RuntimeEngine** (backend/runtime/engine.py) is the central coordinator for AI model providers. It maintains a registry of available providers, manages the currently loaded provider instance, and provides methods for loading, unloading, and switching between providers. The engine initializes during application startup in the FastAPI lifespan function. It supports lazy model loading (controlled by the LAZY_MODEL_LOADING config) and automatic unloading after job completion (AUTO_UNLOAD_AFTER_JOB). The engine delegates to the provider registry for provider

instantiation and to the VRAM tracker for GPU memory allocation.

The **RuntimeInstaller** (backend/runtime/installer.py) handles the complete model installation pipeline: system verification (Python, GPU, disk space), git repository cloning, per-model virtual environment creation via uv, dependency installation, weight file downloading from HuggingFace, and manifest validation. It exports provider metadata (PROVIDER_METADATA), repository configurations (REPOS), supported output formats, render qualities, resolutions, rigging providers, and texture models. The get_install_status() method returns a comprehensive snapshot of all models and their installation state.

8.2 VRAM Allocation Tracker

The **VRAMAllocationTracker** (backend/app/core/managers/vram_tracker.py) is a Redis-backed singleton that manages GPU memory allocation across all AI model providers. It implements an LRU (Least Recently Used) eviction policy with a configurable 2GB safety margin. When a new model requests allocation and insufficient VRAM is available, the tracker automatically evicts the oldest allocated model to free memory. All allocation and eviction events are recorded in the VramAuditLog database table for forensic analysis. The 30-second periodic VRAM health worker (vram_health_worker.py) monitors pressure levels: at 90% utilization it triggers eviction, and at 95% it marks the system as "constrained" in Redis metrics. The tracker uses Redis hashes to store allocation state, enabling multi-worker coordination in distributed deployments.

8.3 Per-Model Virtual Environment System

Each AI model is installed in its own isolated Python virtual environment under **third_party//.venv/**. The **_add_model_env()** function in **backend/app/core/providers/base.py** is the critical integration point: it prepends the model repository path and the model venv site-packages directory to sys.path, then reloads shared packages (huggingface_hub, transformers, diffusers, pydantic, requests, etc.) to prevent version conflicts between the main application and the model-specific dependencies. This system was introduced in v3.2.0 to solve the fundamental problem of different AI models requiring incompatible versions of PyTorch, Transformers, and other ML libraries. The EXTRA_DEPS mechanism (v3.3.1) allows individual providers to specify additional packages that need reloading after the venv is activated.

9. Model Lifecycle

This section documents the complete lifecycle of an AI model in the system, from initial discovery through installation, loading, inference, and eventual removal. Each stage is mapped to the specific files, classes, and functions that implement it.

Stage	Files	Classes/Functions	Details
1. Discovery	discover.py, providers/registry.py	list_models(), get_registry()	Multi-provider search: HuggingFace, GitHub, ModelScope, NGC, CivitAI
2. Download Init	download.py API, download_manager/	start_download(), DownloadQueue.create()	Create DB record, set status=pending, calculate chunks
3. Repo Clone	runtime/installer.py	RuntimeInstaller.clone_repo()	Git clone to third_party//repo/ with .installing.lock
4. Venv Creation	runtime/installer.py	RuntimeInstaller.install_repo_deps()	uv venv + uv pip install in third_party//.venv/
5. Weight Download	runtime/installer.py	RuntimeInstaller.download_weights()	huggingface_hub to third_party//weights/
6. Registry Update	models/registry.py, model_registry.py	InstalledModel.create(), ModelCapability	DB record: status=ready, manifest, size, venv path
7. Load	runtime/engine.py, providers/registry.py	RuntimeEngine.load_provider(), get_provider()	_add_model_env() sys.path setup, dynamic import
8. GPU Allocation	managers/vram_tracker.py	VRAMAllocationTracker.allocate()	Redis-backed, LRU eviction if insufficient, audit log
9. Inference	workers/tasks.py, providers/*.py	provider.generate(), _HunyuanBase._load_model()	GPU inference: text-to-3D or image-to-3D
10. Export	workers/tasks.py, blender/pipeline.py	Blender post-processing, export to GLB/OBJ/FBX	Optional: texture, rigging, LOD, thumbnail
11. Unload	runtime/engine.py, vram_tracker.py	engine.unload_provider(), tracker.deallocate()	Release GPU memory, remove from Redis
12. Repair	installation_workers.py, installer.py	repair_model(), RuntimeInstaller.verify_installation()	Re-verify files, deps, weights; re-run install if needed
13. Delete	models_api.py, runtime/installer.py	remove_model(), shutil.rmtree()	Delete third_party//, remove DB record

Table 9.1 - Complete Model Lifecycle

10. API Documentation

This section documents every REST API endpoint in the system. All endpoints are proxied through the Next.js API route at `/api/v1/*` to the Python backend. The standard response format is: `{"success": true/false, "data": {}, "message": "", "errors": null}`. SSE (Server-Sent Events) endpoints use text/event-stream content type for real-time progress streaming.

Method	Endpoint	Handler File	Purpose
POST	/api/v1/generation	generation.py	Create new 3D generation job
GET	/api/v1/generation/history	generation.py	Paginated job history
GET	/api/v1/generation/cost-estimate	generation.py	Credit cost estimation
GET	/api/v1/generation/{job_id}/status	generation.py	Poll job status + progress
GET	/api/v1/jobs	jobs.py	List all jobs
GET	/api/v1/jobs/{job_id}	jobs.py	Get job details
DELETE	/api/v1/jobs/{job_id}	jobs.py	Delete job + files
GET	/api/v1/runtime/status	runtime.py	Runtime engine status
GET	/api/v1/runtime/health	runtime.py	Runtime health check
GET	/api/v1/runtime/options	runtime.py	Available models, GPUs, VRAM limits
GET	/api/v1/runtime/providers	runtime.py	List all providers
POST	/api/v1/runtime/provider	runtime.py	Switch active provider
POST	/api/v1/runtime/install	runtime.py	Install model (background)
GET	/api/v1/runtime/install/stream	runtime.py	SSE: install progress
POST	/api/v1/runtime/verify	runtime.py	Verify model installation
POST	/api/v1/runtime/restart	runtime.py	Restart runtime engine
POST	/api/v1/runtime/clear-cache	runtime.py	Clear runtime cache
POST	/api/v1/runtime/clear-vram	runtime.py	Clear GPU VRAM
GET	/api/v1/health	health.py	System health (DB+Redis+storage+runtime)
POST	/api/v1/upload/image	upload.py	Upload image (PNG/JPG/WEBP, 50MB)
POST	/api/v1/upload/model	upload.py	Upload 3D model (GLB/GLTF, 100MB)
GET	/api/v1/upload/uploads/{filename}	upload.py	Serve uploaded file

Method	Endpoint	Handler File	Purpose
POST	/api/v1/rigging/anigen-rig	rigging.py	Trigger AniGen rigging task
GET	/api/v1/rigging/{task_id}/status	rigging.py	Poll rigging status
POST	/api/v1/project/export	project.py	Export project with layer toggles
GET	/api/v1/pipelines	pipelines.py	Capability matrix snapshot
GET	/api/v1/pipelines/workspace-models	pipelines.py	Models filtered by workspace
POST	/api/v1/pipelines/{model_id}/toggle	pipelines.py	Enable/disable pipeline feature
GET	/api/v1/discover/models	discover.py	Multi-provider model search
GET	/api/v1/discover/models/{model_id}	discover.py	Get model details
GET	/api/v1/discover/providers	discover.py	List download providers
GET	/api/v1/discover/featured	discover.py	Curated featured models
GET	/api/v1/admin/overview	admin.py	System overview dashboard
GET	/api/v1/admin/logs	admin.py	Get admin logs (paginated)
GET	/api/v1/admin/logs/stream	admin.py	SSE: live log streaming
POST	/api/v1/admin/terminal	admin.py	Execute terminal command
GET	/api/v1/admin/models	admin.py	List all models
POST	/api/v1/admin/models/action	admin.py	Model action (install/uninstall/load)
GET	/api/v1/admin/models/install/stream	admin.py	SSE: install progress
GET	/api/v1/admin/queue/status	admin.py	Celery queue status
POST	/api/v1/admin/queue/purge	admin.py	Purge Celery queues
GET	/api/v1/system/info	system.py	System information
GET	/api/v1/system/compatibility	system.py	Compatibility check (VRAM/CUDA/disk)
GET	/api/v1/system/gpu	system.py	GPU information
GET	/api/v1/system/statistics	system.py	CPU/RAM/disk/GPU statistics
GET	/api/v1/models	models_api.py	List all models

Method	Endpoint	Handler File	Purpose
GET	/api/v1/models/installed	models_api.py	List installed models
GET	/api/v1/models/{model_id}	models_api.py	Get model details
DELETE	/api/v1/models/{model_id}	models_api.py	Delete model + files
POST	/api/v1/models/{model_id}/repair	models_api.py	Repair model installation
GET	/api/v1/models/{model_id}/health	models_api.py	Model health check
GET	/api/v1/download/queue	download.py	Get download queue
POST	/api/v1/download/start	download.py	Start download
POST	/api/v1/download/start-batch	download.py	Start batch download
POST	/api/v1/download/{id}/pause	download.py	Pause download
POST	/api/v1/download/{id}/resume	download.py	Resume download
POST	/api/v1/download/{id}/cancel	download.py	Cancel download
GET	/api/v1/hf-token/status	hf_token.py	Get HF token status
POST	/api/v1/hf-token/set	hf_token.py	Set HF token
POST	/api/v1/hf-token/verify	hf_token.py	Verify HF token
DELETE	/api/v1/hf-token	hf_token.py	Remove HF token
GET	/api/v1/settings/appearance	settings.py	Get appearance settings
PUT	/api/v1/settings/appearance	settings.py	Update appearance settings
GET	/api/v1/settings/workspace	settings.py	Get workspace config
PUT	/api/v1/settings/workspace	settings.py	Update workspace config

Table 10.1 - Complete API Endpoint Reference

11. Frontend Architecture

The frontend is a Next.js 16 application using React 19 with the App Router pattern. It features a sophisticated theming system with 10 presets, a layered project model, real-time 3D visualization, and comprehensive state management through Zustand stores with localStorage persistence.

11.1 Pages and Routing

The application uses Next.js App Router with the following pages: `/` (home, renders `WorkspaceShell`), `/workspace` (also renders `WorkspaceShell`), `/settings` (unified settings with 17 sections organized into 5 groups), `/admin` (deprecated, redirects to `/settings?section=monitoring`), `/generate` (redirects to `/workspace`), `/texture` (`TextureShell` - PBR material editor, mock), `/render` (`RenderShell` - render studio, mock). The API proxy routes at `/api/v1/[...path]` and `/static/[...path]` handle all backend communication.

11.2 State Management

The state management follows a **single source of truth pattern**. The **useAppStore** is the primary Zustand store persisted to localStorage under the key "ai3d:app-state". It contains: generation config (mode, prompt, quality, model, steps, CFG scale, seed), UI state (sidebar collapses, viewer mode, mobile menus), tasks (generation, download, install, render, texture, rigging), downloads queue, install progress, project (current project with composable layers), and filter/tab state. Three proxy stores provide domain-specific APIs: **useUIStore** (viewer, capabilities, mobile), **useGenerationStore** (config construction, history loading), and **useProjectStore** (layer queries, enabled/visible filters). The **useThemeStore** is a separate persisted store for appearance configuration with 10 presets, each defining accent colors, glass effects, border radius, button styles, lighting effects, 13 animation toggles, animation speed, easing, font size, density, theme mode, and navbar style. The **AppearanceProvider** component subscribes to the theme store and writes 50+ CSS custom properties to `document.documentElement`.

11.3 Component Hierarchy

Component	Parent	Children/Dependencies	Purpose
WorkspaceShell	page.tsx	CreativeWorkspaceLayout, ThreeDViewer, CommandPalette	Main workspace container
CreativeWorkspaceLayout	WorkspaceShell	12 sidebar tabs, 3D viewer, history grid	Primary workspace layout
ThreeDGenerationTab	CreativeWorkspaceLayout	ModelSelector, PromptInput, Canvas, HistoryGrid	Main generation UI (69KB)
ThreeDViewer	WorkspaceShell	ViewerScene, ViewerToolbar, ViewerPlaceholder	React Three Fiber 3D viewer

Component	Parent	Children/Dependencies	Purpose
ViewerScene	ThreeDViewer	OrbitControls, Grid, Environment, Float	Three.js scene setup
PromptInput	ThreeDGenerationTab	Textarea, suggestions, image upload	Text prompt entry
ModelSelector	ThreeDGenerationTab	Select, useWorkspaceModels	AI model selection
GenerateButton	ThreeDGenerationTab	useGeneration hook	Trigger generation
JobProgressMonitor	ThreeDGenerationTab	Stage indicators, progress bar	Real-time progress
ExportDialog	ThreeDGenerationTab	Format selection, download buttons	GLB/OBJ/FBX export
AppearanceProvider	layout.tsx	useThemeStore, CSS custom properties	Theme system (50+ CSS vars)
Providers	layout.tsx	QueryClientProvider, Toaster	Root React providers
CommandPalette	WorkspaceShell	shadcn Command, Framer Motion	Cmd+K command palette
AnimatedBackground	Various pages	Gradient blobs, ParticleCanvas	Decorative background

Table 11.1 - Component Hierarchy

11.4 API Client Architecture

The API client (**services/apiClient.ts**) provides a comprehensive HTTP communication layer with: (1) base request method with 15s timeout and JSON parsing; (2) exponential backoff retry ($100\text{ms} \times 2^{\text{attempt}} + \text{jitter}$, max 3 retries, skip on 4xx except 429); (3) circuit breaker pattern racing requests against 5s timeout; (4) GET cache awareness via CacheService (60s TTL, 100 max entries, LRU eviction); (5) FormData upload without Content-Type header (browser sets multipart boundary); (6) EventSource SSE streaming with cleanup function. The **API_URL** is empty by default (relative URLs for Docker proxy), falling back to **NEXT_PUBLIC_API_URL**. All services (generation, runtime, admin, upload) build on this client.

12. Docker and Deployment Architecture

AI 3D Studio is primarily deployed natively using shell scripts, with Docker packaging as a secondary path. The **package-production.sh** script captures a prepared machine state into a production Docker image. There is no Docker Compose file in the repository - the primary deployment uses **manager.sh** and the `scripts/` directory for service lifecycle management.

12.1 Native Deployment (Primary)

The native deployment path consists of: (1) **scripts/setup.sh** (701 lines) - full system installation supporting Ubuntu 20.04/22.04/24.04, installs Python 3.12, uv, Node.js 20, PostgreSQL 16 + pgvector, Redis 7, Blender 4.2.3, creates project directories, installs Python and Node.js dependencies; (2) **scripts/start.sh** (444 lines) - starts all services in order (PostgreSQL > Redis > Alembic migrations > Backend API > Celery Worker > Frontend), with SQLite fallback and cloud environment auto-bootstrap; (3) **scripts/stop.sh** (93 lines) - graceful shutdown in reverse order with 10s TERM grace period; (4) **manager.sh** (398 lines) - interactive TUI with 13-item menu for service management, health checks, database operations, log viewing, and Cloudflare tunnel management.

12.2 Docker Packaging (Secondary)

The **package-production.sh** (454 lines) generates a production Docker image with: CUDA 12.1 base, Miniconda Python 3.12, Node.js 22, and relinked per-model venvs. The **next.config.ts** sets output to standalone for Docker-friendly builds. The **scripts/install_nvidia_toolkit.sh** (128 lines) handles NVIDIA Container Toolkit installation for GPU passthrough. The **scripts/cloudflare.sh** (174 lines) manages Cloudflare tunnels for remote access. The **scripts/colab.sh** provides Google Colab-specific bootstrap with SQLite mode and user-space Redis.

12.3 Environment Variables

Variable	Default	Purpose	Critical ?
BACKEND_URL	http://localhost:8000	Python backend URL (read at request time)	Yes
DATABASE_URL	postgresql+asyncpg://...	Primary database connection	Yes
REDIS_URL	redis://localhost:6379/0	Redis for Celery + pub/sub	Yes
CELERY_BROKER_URL	redis://localhost:6379/0	Celery message broker	Yes
AI_PROVIDER	triposr	Default AI provider	Yes
RUNTIME_MODE	local	local or api mode	Yes

Variable	Default	Purpose	Critical ?
CUDA_DEVICE	auto	GPU device selection	Yes
MAX_VRAM_MB	0 (auto)	Maximum VRAM allocation	Yes
STORAGE_BACKEND	local	local or s3	Medium
STORAGE_LOCAL_PATH	./backend/storage	Local storage root	Medium
BLENDER_EXECUTABLE	blender	Blender binary path	Medium
BLENDER_ENABLED	true	Enable Blender post-processing	Medium
MAX_CONCURRENT_JOBS	1	Max parallel generation jobs	High
OPENAI_API_KEY	(empty)	For prompt enhancement (optional)	Low
GITHUB_TOKEN	(empty)	GitHub API rate limit bypass	Low
NVIDIA_NGC_API_KEY	(empty)	NVIDIA model catalog access	Low
GEMINI_API_KEY	(empty)	Google Gemini API (metadata)	Low

Table 12.1 - Environment Variables

13. Startup Sequence

The complete startup sequence traces the system from the initial shell script invocation to a fully operational platform. This includes service dependencies, health verification, and fallback mechanisms.

```
sequenceDiagram
    participant User as Operator
    participant MGR as manager.sh
    participant SS as scripts/start.sh
    participant PG as PostgreSQL
    participant RD as Redis
    participant AL as Alembic
    participant BE as FastAPI Backend
    participant CE as Celery Worker
    participant FE as Next.js Frontend

    User->>MGR: Select "Start all services"
    MGR->>SS: Execute scripts/start.sh
    SS->>PG: Start PostgreSQL (or SQLite fallback)
    PG->>SS: Connection OK
    SS->>RD: Start Redis (or in-process fallback)
    RD->>SS: PONG
    SS->>AL: alembic upgrade head
    AL->>SS: Migrations applied
    SS->>BE: uvicorn backend.app.main:app --port 8000
    BE->>BE: Normalize CUDA_VISIBLE_DEVICES
    BE->>BE: log_startup_diagnostics()
    BE->>BE: Ensure storage directories
    BE->>BE: Initialize RuntimeEngine
    BE->>PG: Test DB connection + create tables
    BE->>RD: Test Redis connection
    BE->>SS: Health endpoint returns 200
    SS->>CE: celery -A workers.celery_app worker
    CE->>CE: Normalize CUDA (before torch import)
    CE->>CE: Verify GPU availability
    CE->>RD: Connect to broker
    CE->>SS: Worker ready
    SS->>FE: next start (or standalone server)
    FE->>SS: Listening on :3000
    SS->>MGR: All
```

```
services started MGR-->User: Ready
```

Diagram 13.1 - Startup Sequence

14. Generation Pipeline

The generation pipeline is the core value proposition of AI 3D Studio. It transforms a text prompt or reference image into a complete 3D model with optional textures, rigging, and rendering. This section traces every function call in the pipeline from input to final delivery.

```
flowchart TD
  A[User Input: Text Prompt / Image Upload] --> B{Input Type?}
  B -->|Text| C[Validate prompt length 2000 chars]
  B -->|Image| D[Upload via uploadService]
  D --> E[Validate: PNG/JPG/WEBP, 256-8192px, 50MB]
  E --> F[POST /api/v1/generation]
  F --> G[Create GenerationJob DB record]
  G --> H[Enqueue Celery: generate_3d_model]
  H --> I{Prompt Enhancement?}
  I -->|Yes| J[OpenAI GPT enhance prompt]
  I -->|No| K[Use original prompt]
  J --> L[Load provider: get_provider]
  K --> L
  L --> M[_add_model_env: venv sys.path setup]
  M --> N[VRAM Tracker: allocate GPU memory]
  N --> O{VRAM sufficient?}
  O -->|Yes| P[provider.generate]
  O -->|No| Q[LRU evict oldest model]
  Q --> N
  P --> R[GPU Inference: mesh generation]
  R --> S{Generate Texture?}
  S -->|Yes| T[Texture generation pass]
  S -->|No| U[Raw mesh output]
  T --> V[Auto Rig?]
  U --> V
  V -->|Yes| W[AniGen rigging pipeline]
  V -->|No| X[Skip rigging]
  W --> Y[Blender post-processing]
  X --> Y
  Y --> Z[Export to GLB/OBJ/FBX]
  Z --> AA[Render thumbnail via Trimesh]
  AA --> AB[Compute mesh stats: poly/vertex count]
  AB --> AC[Update job: status=completed]
  AC --> AD[Unload provider, deallocate VRAM]
  AD --> AE[Frontend receives result via poll/SSE]
  AE --> AF[Display in 3D viewer + history]
```

Diagram 14.1 - Complete Generation Pipeline

15. Error Propagation

This section traces how errors originate, propagate through the system, are caught, and presented to the user. The error handling strategy follows a layered approach: catch locally, enrich with context, propagate upward, and present actionable messages at the UI layer.

Error	Origin	Propagation	Caught By	User Response
Invalid prompt	hooks/useGeneration.ts	Frontend validation	Frontend	Toast: prompt required
Invalid image	services/uploadService.ts	validateFile()	Frontend	Toast: format/size error
Backend unreachable	services/apiClient.ts	request() timeout	Frontend	Toast: connection error

Error	Origin	Propagation	Caught By	User Response
DB connection failure	backend/app/main.py	lifespan()	Startup	Server fails to start
Redis connection failure	backend/app/main.py	lifespan()	Startup	Server starts degraded
Provider not found	core/providers/registry.py	get_provider()	Celery task	Job status=failed
VRAM insufficient	managers/vram_tracker.py	allocate()	Celery task	Auto-evict or fail
CUDA OOM	providers/hunyuan3d_local.py	GPU inference	Provider try/except	Job status=failed
Model load failure	providers/hunyuan3d_local.py	_load_model()	Provider try/except	Job status=failed
Blender timeout	core/blender/pipeline.py	subprocess call	Worker	Job status=failed
Download failure	core/download_manager/	downloader	Download worker	Status=failed, retry
Git clone failure	runtime/installer.py	clone_repo()	Install task	Status=failed
pip install failure	runtime/installer.py	install_repo_deps()	Install task	Status=failed
Pydantic validation	backend API routes	request validation	FastAPI	422 response
Auth failure (HF)	api/v1/hf_token.py	HfApi.whoami()	API route	401 status
File not found	api/v1/upload.py	Path traversal check	API route	404 response
Terminal command blocked	api/v1/admin.py	Security filter	API route	403: command blocked
SSE connection drop	services/adminService.ts	EventSource error	Frontend	Silent reconnect

Table 15.1 - Error Propagation Map

16. Debugging Guide

16.1 Download Failures

Symptoms: Model download stalls, fails with network error, or shows incorrect progress.

Key Files: backend/app/core/download_manager/downloader.py, backend/app/core/downloader/mirror_fallback.py, backend/app/api/v1/download.py

Key Functions: download_model(), download_with_mirrors(), resume_download()

Common Causes: (1) HuggingFace rate limiting without API token - set HF_TOKEN in .env; (2) Network proxy/firewall blocking - check network connectivity; (3) Insufficient disk space - check df -h; (4) Checksum mismatch - file corrupted during download, re-download; (5) Git LFS not installed - model weights require git-lfs for clone.

Solutions: Set GITHUB_TOKEN and HF_TOKEN for higher rate limits. Use scripts/update-models.sh --verify to validate all downloads. Check storage/ and third_party/ disk usage. Use the admin DownloadsTab to monitor active downloads and retry failed ones.

16.2 GPU / CUDA Issues

Symptoms: CUDA out of memory, GPU not detected, inference fails with CUDA error.

Key Files: backend/runtime/gpu.py, backend/app/core/managers/vram_tracker.py, backend/app/workers/celery_app.py

Key Functions: get_gpu_info(), get_vram_usage(), allocate(), empty_cuda_cache()

Common Causes: (1) CUDA_VISIBLE_DEVICES not normalized - celery_app.py normalizes this, but check if set incorrectly before app loads; (2) PyTorch version mismatch with CUDA driver - ensure CUDA 12.1 compatibility; (3) Other processes using GPU - check nvidia-smi for VRAM usage; (4) VRAM safety margin too small - increase VRAM_SAFETY_MARGIN_MB; (5) Model loaded but not unloaded - check VRAM tracker for leaks.

Solutions: Run nvidia-smi to verify GPU visibility. Check GET /api/v1/system/gpu for detected devices. POST /api/v1/runtime/clear-vram to free cached GPU memory. Reduce MAX_CONCURRENT_JOBS to 1. Check the VRAM audit log in the admin panel for eviction history.

16.3 API / Backend Issues

Symptoms: 500 errors, API proxy failures, backend not responding.

Key Files: app/api/v1/[...path]/route.ts, backend/app/main.py, backend/app/config.py

Common Causes: (1) BACKEND_URL incorrect - must match Docker service name or localhost; (2) Database migration pending - run alembic upgrade head; (3) Missing .env file - copy from .env.example; (4) Port conflict - check if 8000 or 3000 already in use; (5) Python import error - check pip install.

Solutions: Check dev.log and logs/ directory for stack traces. Verify GET /api/v1/health returns OK. Use manager.sh option 5 to check all service statuses. Use admin TerminalTab to run diagnostic

commands.

16.4 Worker / Celery Issues

Symptoms: Jobs stuck in queued status, worker not processing, health checks failing.

Key Files: backend/app/workers/celery_app.py, backend/app/workers/tasks.py, scripts/start.sh

Common Causes: (1) Celery worker not started - check manager.sh status; (2) Redis broker unreachable - verify Redis is running on port 6379; (3) CUDA import fails in worker - check celery_app.py CUDA normalization; (4) Task not registered - celery_app.py explicitly imports all task modules; (5) Worker crashed - check logs.

Solutions: Use admin QueueTab to inspect Celery queues. POST /api/v1/admin/queue/purge to clear stuck tasks. Check worker logs via manager.sh option 6. Verify worker heartbeat in Redis.

16.5 Frontend Issues

Symptoms: Blank page, hydration errors, 3D viewer not loading, state not persisting.

Key Files: app/layout.tsx, app/api/v1/[...path]/route.ts, stores/useAppStore.ts, features/workspace/viewer/ThreeDViewer.tsx

Common Causes: (1) Hydration mismatch - AppearanceProvider writes CSS vars on client only; (2) API proxy timeout - increase timeout in route.ts; (3) ThreeDViewer SSR - dynamically imported with ssr: false; (4) localStorage quota exceeded - clear ai3d:app-state and ai-studio-theme; (5) Build stale - delete .next and rebuild.

Solutions: Check browser console for errors. Clear localStorage and refresh. Verify API proxy is forwarding requests correctly. Check React Query devtools for failed queries. Disable browser extensions that may interfere with WebGL.

17. Dead Code Analysis

This section identifies unused files, folders, classes, functions, APIs, and dependencies in the codebase. Dead code increases maintenance burden, confuses new developers, and may contain security vulnerabilities.

Item	Type	Location	Reason	Recommendation
vite (devDep)	Dependency	package.json	Project uses Next.js, not Vite	Remove from package.json
@tailwindcss/vite	Dependency	package.json	Vite plugin, unused in Next.js	Remove from package.json

Item	Type	Location	Reason	Recommendation
@vitejs/plugin-react	Dependency	package.json	Vite React plugin, unused	Remove from package.json
@types/jquery	Dependency	package.json	No jQuery in project	Remove from package.json
framer-motion + motion	Dependency	package.json	Same package (rebranded), dual install	Keep only framer-motion
firebase-tools	Dependency	package.json	No Firebase usage found in code	Remove or document purpose
.eslintrc.json	Config file	Root	Superseded by eslint.config.mjs	Delete
app/admin/page.tsx	Page	app/admin/	Deprecated, redirects to /settings	Delete or remove redirect
app/generate/page.tsx	Page	app/generate/	Just redirects to /workspace	Use route redirect instead
features/admin/AdminShell.tsx	Component	features/admin/	Legacy, merged into settings page	Remove entire admin/feature
features/texture/TextureShell.tsx	Component	features/texture/	Mock implementation, no backend	Implement or remove
features/render/RenderShell.tsx	Component	features/render/	Mock implementation, hardcoded queue	Implement or remove
test_script.py	Script	Root	Prints "Test" only, not a real test	Delete
core/providers/triposf_provider.py	File	backend/	Has syntax error (line 90), mock impl	Fix bug or delete
hooks/use-toast.ts	Hook	hooks/	shadcn/ui compat layer, Sonner used instead	Remove if unused
lib/next-compat-*.tsx (4 files)	Utility	lib/	Framework decoupling shims, possibly unused	Verify usage, remove if not
backend/tests/	Directory	backend/tests/	Only one test file, minimal coverage	Expand or acknowledge

Table 17.1 - Dead Code Inventory

18. Refactoring Opportunities

18.1 Architecture Smells

admin.py is 1811 lines: The admin API file is the largest in the codebase and handles logging, terminal, download progress, system overview, model management, and queue management. It should be split into separate route modules following the single-responsibility principle. The logging system (2000-entry ring buffer + file sink + SSE) and the terminal system (command execution with security filtering) are independent concerns that deserve their own files.

ThreeDGenerationTab.tsx is 69KB: This is the largest frontend component and handles model selection, prompt input, quality settings, 3D preview with React Three Fiber, generation progress, history management, layer controls, export, and image upload. It should be decomposed into smaller, focused components following the container/presentational pattern.

18.2 Tight Coupling

Backend core depends on runtime at module level: The provider registry in `core/providers/registry.py` imports from `runtime.installer` at module level (mitigated by `lru_cache`). This creates a conceptual circular dependency between the core business logic layer and the runtime infrastructure layer. A better approach would be to inject the install status as a parameter rather than importing it directly.

Frontend stores have implicit coupling: All proxy stores (`useUIStore`, `useGenerationStore`, `useProjectStore`) silently depend on `useAppStore`. If `useAppStore` changes its state shape, all proxy stores break without any type safety. A more explicit approach would use Zustand slices with type-safe selectors.

18.3 Code Duplication

API response normalization: Multiple services (`generationService`, `runtimeService`, `adminService`) implement similar response normalization patterns (`data?.data ?? data ?? {}`). This should be extracted into a shared utility in `apiClient.ts`.

Provider stubs: Six provider files (`hunyuan3d.py`, `trellis.py`, `triposr.py`, `instant_mesh.py`, `triposg_provider.py`, `triposf_provider.py`, `unirig_provider.py`, `holopart_provider.py`, `detailgen3d.py`) are mock or stub implementations that follow a similar pattern. A `MockProviderBase` class could reduce this duplication.

18.4 Hidden State

_AdminLogHandler in-memory state: The admin log system maintains a 2000-entry ring buffer and SSE queues as module-level global state in `admin.py`. This state is not persisted across restarts and is invisible to monitoring tools. It should be replaced with a proper logging framework handler.

_DL_STATE in admin.py: Download progress tracking uses a module-level dictionary with file-backed persistence. This global mutable state is prone to race conditions in multi-worker deployments and should be moved to Redis or the database.

18.5 Recommended Improvements

1. **Split admin.py** into 5+ smaller route modules (logs, terminal, overview, models, queue). 2. **Decompose ThreeDGenerationTab** into GenerationForm, GenerationPreview, GenerationProgress, GenerationHistory components. 3. **Add authentication** - the app has no auth system despite having admin endpoints and terminal access. 4. **Replace mock providers** with real implementations or remove them entirely. 5. **Add end-to-end tests** - the test coverage is minimal (one test file). 6. **Fix known bugs:** response.py syntax error (line 15), triposf_provider.py syntax error (line 90). 7. **Consolidate animation libraries** - remove duplicate motion package. 8. **Implement texture and render shells** - currently mock-only. 9. **Add WebSocket** for real-time bidirectional communication instead of SSE + polling. 10. **Add OpenAPI schema generation** for the frontend API client.

19. Security Review

This section identifies security concerns in the codebase, categorized by severity.

Issue	Severity	Location	Details	Recommendation
No authentication	Critical	Entire backend	All API endpoints are publicly accessible, including admin panel, terminal execution, and model management	Add NextAuth.js or similar auth system
Terminal command execution	Critical	admin.py	POST /api/v1/admin/terminal allows arbitrary command execution with basic blocklist (rm -rf /, sudo, curl bash)	Implement whitelist-based approach, not blocklist
Path traversal in upload	High	upload.py	Path traversal protection exists but relies on os.path.basename - verify against symlink attacks	Use pathlib.resolve() and check prefix
Subprocess injection	High	blender/pipeline.py	Blender invoked via subprocess with file paths from user input	Validate all paths, use shlex.quote
Environment variable secrets	Medium	.env, config.py	API keys (OpenAI, GitHub, NVIDIA) stored in .env plaintext	Use secret manager in production

Issue	Severity	Location	Details	Recommendation
No rate limiting	Medium	API layer	No rate limiting on generation or upload endpoints	Add slowapi or nginx rate limiting
CORS wildcard in dev	Medium	main.py	CORS allows all origins in development mode	Restrict in production
SQL injection risk	Low	All DB operations	Mitigated by SQLAlchemy ORM parameterized queries	Ensure no raw SQL with string formatting
Dependency vulnerabilities	Medium	requirements.txt	60+ Python packages, 42+ npm packages - potential CVEs	Run pip-audit and npm audit regularly
File upload DoS	Medium	upload.py	50MB image / 100MB model limits but no per-user quota	Add per-user upload quotas
SSE connection exhaustion	Low	admin.py, generation.py	SSE endpoints hold connections open indefinitely	Add connection timeout and limit

Table 19.1 - Security Review

20. Appendix

20.1 Glossary

Term	Definition
3D Generation	The process of creating a 3D mesh model from a text prompt or reference image using AI inference.
Provider	An AI model implementation that handles inference for a specific 3D generation algorithm (e.g., Hunyuan3D, TRELIS, TripoSR).
Runtime Engine	The backend subsystem that manages provider lifecycle: installation, loading, GPU allocation, inference, and unloading.
VRAM Tracker	Redis-backed GPU memory allocator with LRU eviction, 2GB safety margin, and database audit trail.
Celery	Distributed task queue system used for asynchronous GPU-bound generation jobs and download orchestration.
Per-Model Venv	Isolated Python virtual environment for each AI model under <code>third_party//.venv/</code> to prevent dependency conflicts.

Term	Definition
_add_model_env()	Critical function that prepends a model venv to sys.path and reloads shared packages to activate per-model isolation.
SSE	Server-Sent Events - one-way real-time communication from server to client used for progress streaming and log tailing.
Capability Matrix	Feature gating system that maps model capabilities to workspace types, controlling which features appear in the UI.
Project Layer	Composable unit in the project system (texture, rigging, animation, LOD, segmentation, remesh) that can be independently toggled.
AGENTS.md	AI coding assistant instruction file containing project-specific conventions, debugging patterns, and code quality rules.
uv	Fast Python package manager (Rust-based) used exclusively for all Python dependency management in the project.
AniGen	Character auto-rigging AI model that generates skeletal armatures for 3D character meshes.
GLB	GL Transmission Format Binary - the primary 3D model output format, a binary glTF file.
PBR	Physically Based Rendering - material system using albedo, normal, roughness, metallic, AO, and height maps.
Hunyuan3D	Tencent AI 3D generation model supporting text-to-3D and image-to-3D, requiring 16GB VRAM.
TRELLIS	Microsoft 3D generation model for image-to-3D conversion.
TripoSR	Tripo AI fast single-image-to-3D model requiring approximately 4GB VRAM.
MockProvider	Placeholder provider that writes GLB_PLACEHOLDER bytes, always healthy, used for testing and development.

Table 20.1 - Glossary

20.2 File Cross-Reference Index

The following index maps important files to all sections in this document where they are discussed, enabling quick cross-reference navigation.

File	Sections
backend/app/main.py	2, 3.1, 5.1, 5.2, 15, 16.3, 19
backend/app/config.py	2, 3.1, 12.3
backend/app/database.py	2, 3.1, 5.1
backend/app/api/v1/admin.py	2, 3.2, 4.3, 10, 18.1, 18.4, 19
backend/app/api/v1/generation.py	2, 3.2, 5.2, 10

File	Sections
backend/app/api/v1/runtime.py	2, 3.2, 10
backend/app/core/providers/registry.py	2, 3.3, 4.3, 5.2, 5.4, 6.3, 8.1, 18.2
backend/app/core/providers/base.py	3.3, 5.2, 5.4, 8.3, 9
backend/app/core/providers/hunyuan3d_local.py	3.3, 5.2, 5.4, 9, 15
backend/app/core/managers/vram_tracker.py	2, 3.4, 5.4, 8.2, 9, 15, 16.2
backend/app/workers/tasks.py	2, 3.4, 5.2, 5.4, 7, 9, 14, 15
backend/app/workers/celery_app.py	2, 3.4, 5.1, 16.4
backend/runtime/engine.py	2, 3.4, 5.4, 8.1, 9
backend/runtime/installer.py	2, 3.4, 5.3, 7, 8.1, 9, 16.1
backend/runtime/gpu.py	2, 3.4, 8.2, 16.2
backend/runtime/storage.py	2, 3.4, 5.3, 7, 9
app/api/v1/[...path]/route.ts	2, 3.5, 4.1, 10, 11.4, 16.3, 16.5
stores/useAppStore.ts	2, 3.5, 11.2, 18.2
stores/useThemeStore.ts	2, 3.5, 11.2
services/apiClient.ts	2, 3.5, 11.4
services/generationService.ts	2, 3.5, 5.2, 11.4, 18.3
hooks/useGeneration.ts	2, 3.5, 5.2, 11.1, 15
hooks/useTaskManager.ts	2, 3.5, 11.1
features/workspace/new-ui/CreativeWorkspaceLayout.tsx	2, 3.6, 11.1, 11.3
features/workspace/new-ui/ThreeDGenerationTab.tsx	2, 3.6, 11.1, 11.3, 18.1
features/workspace/viewer/ThreeDViewer.tsx	2, 3.6, 11.1, 11.3, 16.5
scripts/setup.sh	2, 12.1, 13
scripts/start.sh	2, 5.1, 12.1, 13, 16.4
manager.sh	2, 12.1, 13, 16.3
backend/app/core/capability_matrix.py	2, 4.3, 8.1
backend/app/utlis/response.py	2, 3.1, 17, 18.5

Table 20.2 - File Cross-Reference Index

20.3 Version History

Version	Date	Key Changes
v3.3.1	2026-08-07	Fix shared dependency version conflicts in local providers. Enhanced <code>_add_model_env()</code> with <code>EXTRA_DEPS</code> mechanism. Fixed provider map divergence.
v3.3.0	2026-08-06	Workspace compatibility and texture pipeline. Model-to-workspace filtering via capability matrix. New API endpoints. PBR controls.
v3.2.0	2026-01-25	uv-only package management. Per-model isolated environments under <code>third_party/</code> . Weights directory restructure. Install concurrency control.
Pre-3.2	Earlier	Initial architecture. Single shared Python environment for all models. pip-based dependency management. Monolithic model storage.

Table 20.3 - Version History

20.4 Known Bugs

1. backend/app/utils/response.py line 15 - Syntax Error: The `paginated()` function has a syntax error where `"page page,"` should be `"page": page,`. This would cause a runtime error if the function is called.

2. backend/app/core/providers/triposf_provider.py line 90 - Syntax Error: The metadata dictionary has a missing colon: `metadata={"provider "triposf"...}` should be `metadata={"provider": "triposf"...}`. This causes a `SyntaxError` when the module is imported, making the TripoSF provider completely non-functional.

3. Version inconsistency: `package.json` reports version 0.1.0 while `.env.example` reports 3.2.0 and `CHANGELOG` tracks up to 3.3.1. This can cause confusion in deployment and monitoring tools.

4. Two ESLint configurations: Both `.eslintrc.json` (legacy format) and `eslint.config.mjs` (flat config) exist. The legacy file should be removed to avoid confusion.